

DEEP LEARNING PROJECT

GAN-Based Synthetic Handwritten Digit Generation

Training a Generative Adversarial Network to synthesise novel handwritten digits from the MNIST dataset.

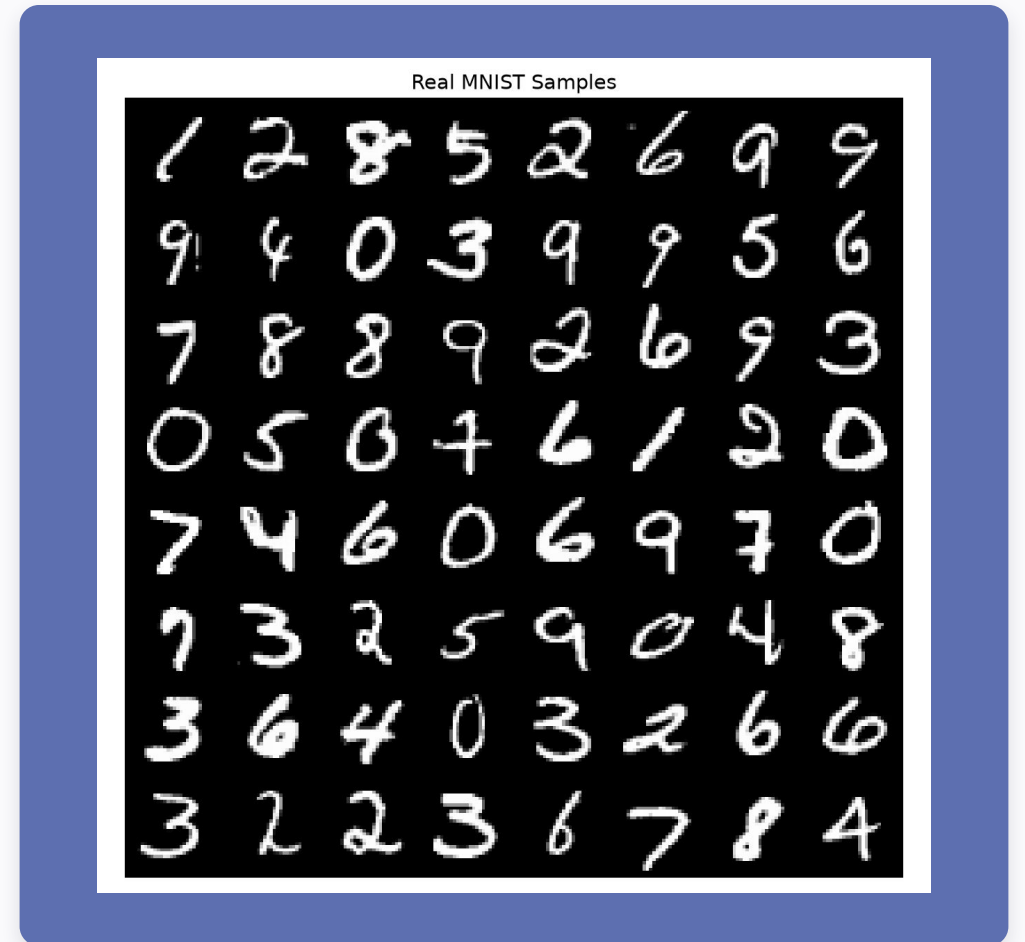
PROJECT TEAM

Amrith Saras · **Annabel Marianne Victor** · **Diya Jothish**

102903 / CO701B Deep Learning | Rajagiri School of Engineering & Technology

Problem Statement

- *Handwritten digit recognition is a well studied classification task with mature, high accuracy solutions.*
- *Generating realistic synthetic digits from random noise is a fundamentally different and harder problem: The model must learn the underlying data distribution, not just a decision boundary.*
- *This project implements a Generative Adversarial Network trained on MNIST to learn that distribution and produce novel, visually plausible handwritten digit images.*



Real MNIST handwritten digits: the target distribution

Objectives



Understand the GAN framework: The competing objectives of Generator and Discriminator.



Implement a GAN with two competing networks, trained end to end in PyTorch.



Train the model on the full MNIST dataset- all 60,000 greyscale handwritten digit images.



Generate novel synthetic handwritten digits from random Gaussian noise.

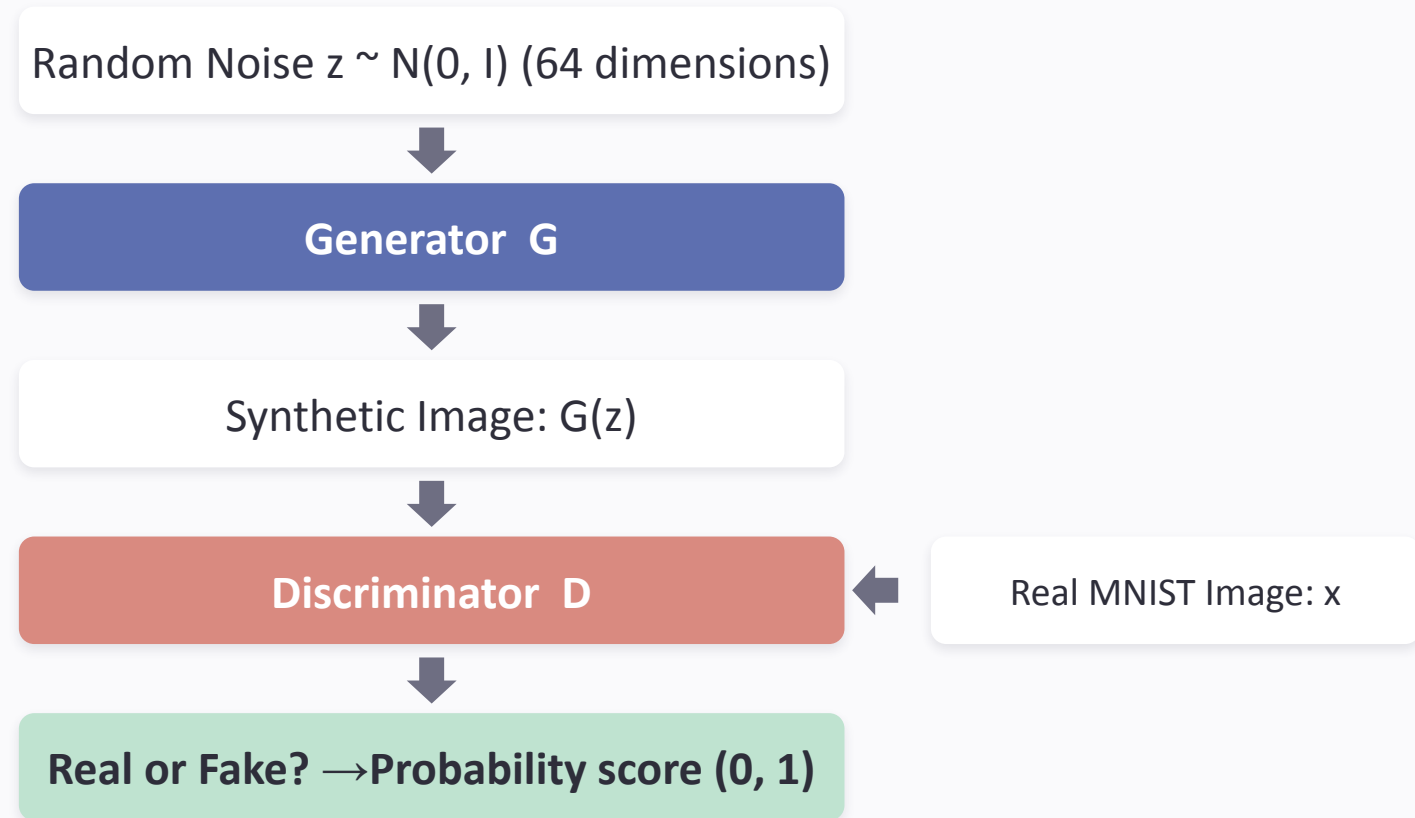


Analyse generation quality through training dynamics and visual inspection.



Explore the learned latent space through interpolation between noise vectors.

The GAN Idea: Two Networks, One Contest



Training objective: $\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$

At Nash equilibrium, the Generator matches the real data distribution and $D(x)$ approaches 0.5 for every input, so it can no longer tell real from generated.

Dataset: MNIST

Property	Value
<i>Name</i>	<i>MNIST</i>
<i>Source</i>	<i>torchvision.datasets.MNIST</i> (auto-download)
<i>Channels</i>	<i>1 (greyscale)</i>
<i>Pixel normalisation</i>	$[0, 1] \rightarrow [-1, 1]$

**60,000**

Training images

**10,000**

Test images

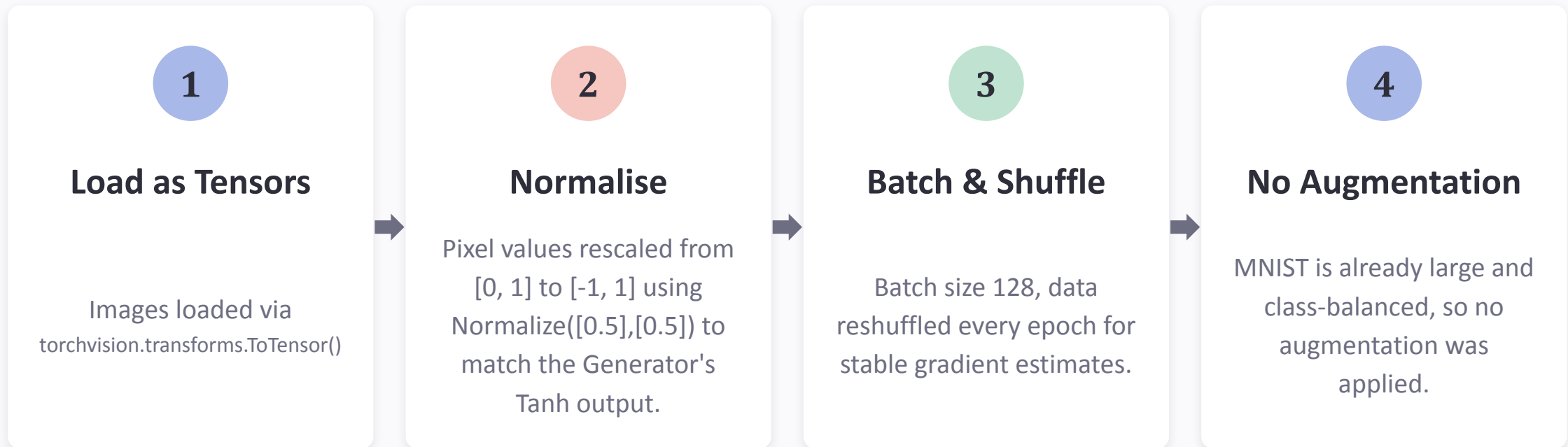
 **28×28**

Pixel resolution

**10**

Digit classes (0–9)

Preprocessing Pipeline



Methodology: Why a Fully-Connected GAN

Architecture choice

- *This implementation uses a fully-connected (MLP) GAN: Both the Generator and Discriminator are stacked Linear layers with Leaky ReLU activations, rather than convolutional layers.*
- *This keeps the model straightforward to implement, train and reason about, while still performing well on small 28×28 greyscale images such as MNIST digits.*

LeakyReLU (slope 0.2)



Used in every hidden layer of both networks. Avoids the dead neurons that plain ReLU can cause when inputs go negative.

Tanh at Generator output



Squashes pixel values to $[-1, 1]$, matching the normalised input images.

Sigmoid at Discriminator output



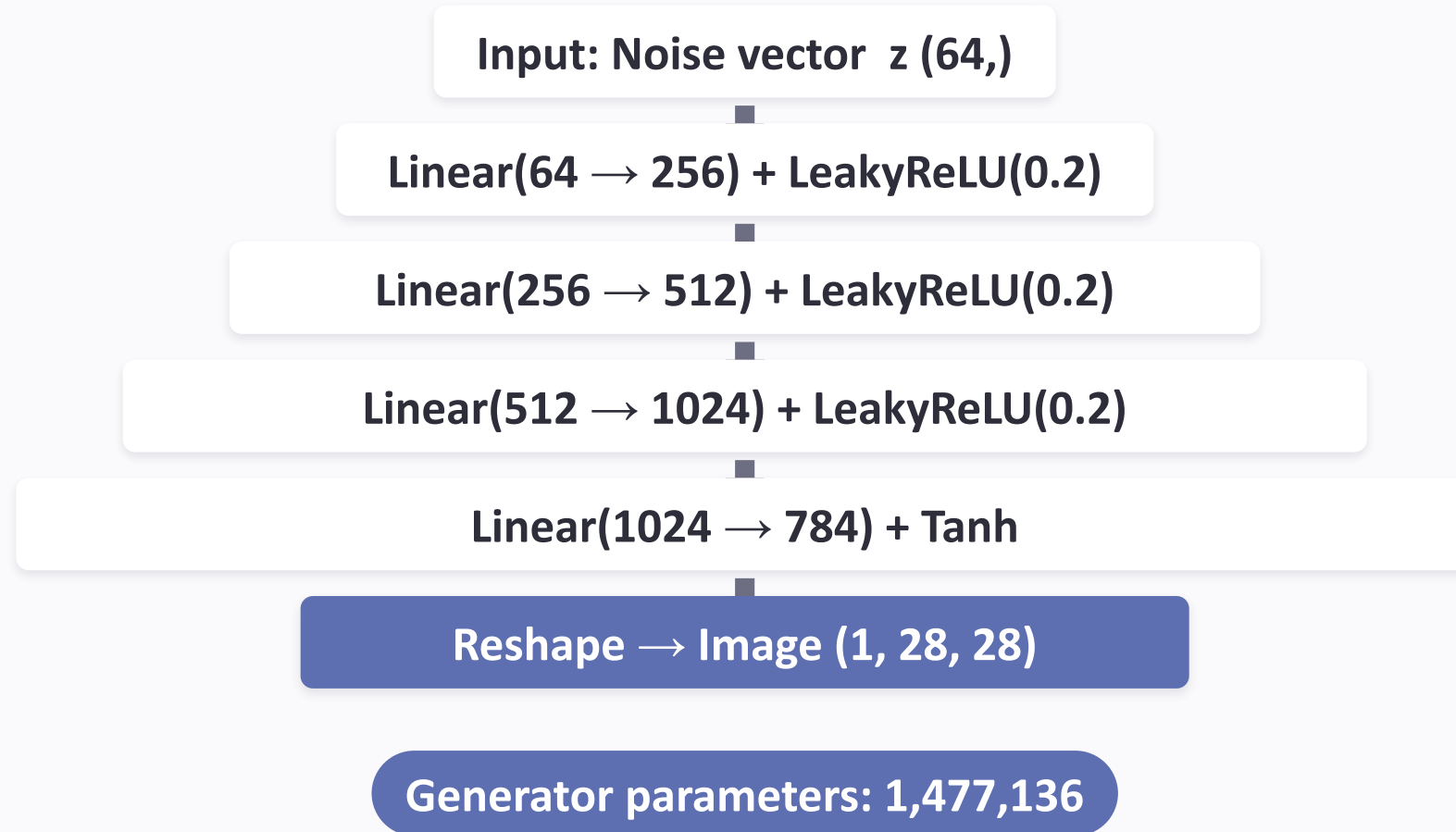
Produces a clean probability in $(0, 1)$ for the real / fake decision.

Flatten & reshape

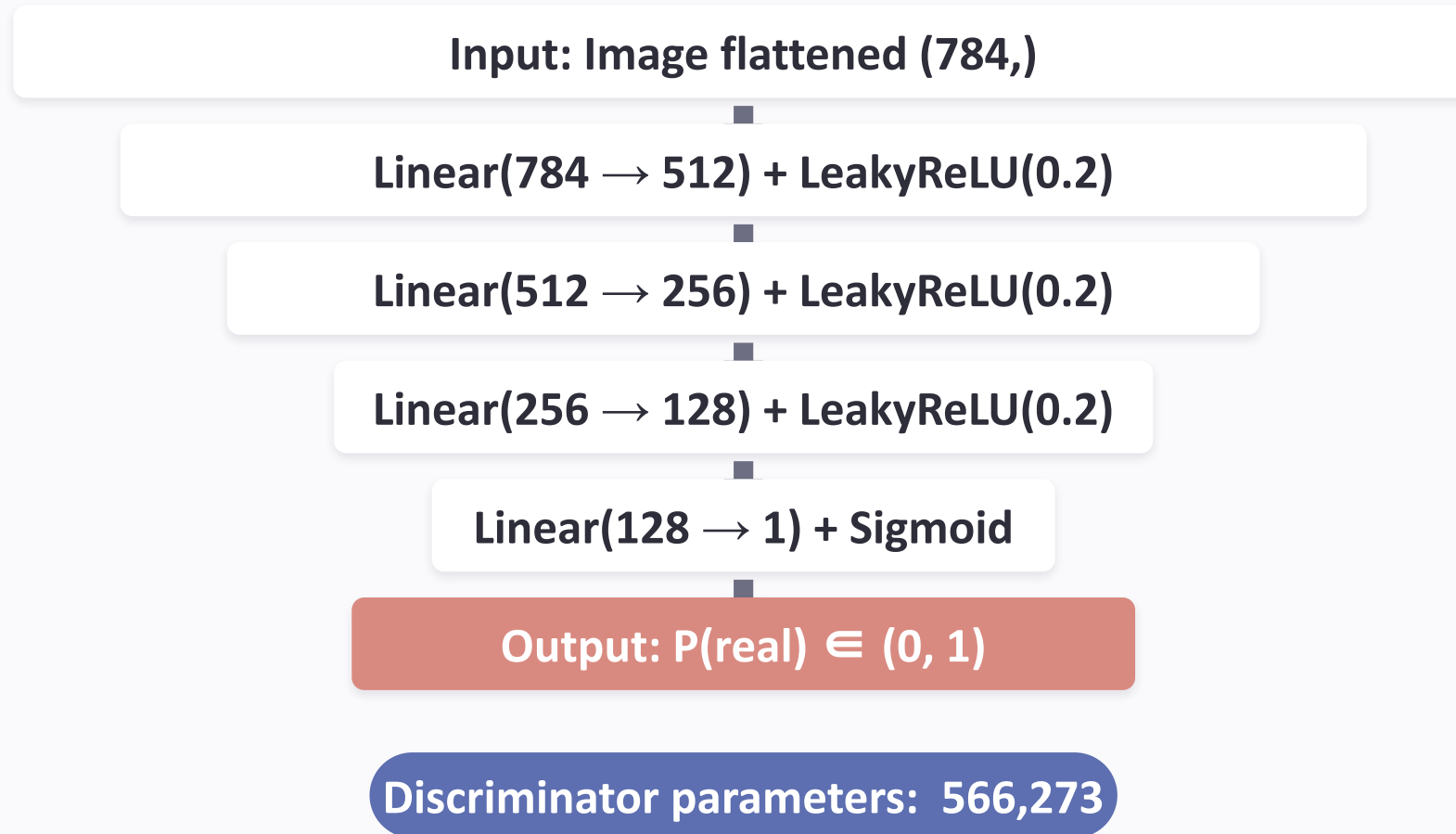


Images are flattened to 784-dim vectors for the Discriminator, and reshaped back to 28×28 at the Generator output.

Generator Architecture



Discriminator Architecture



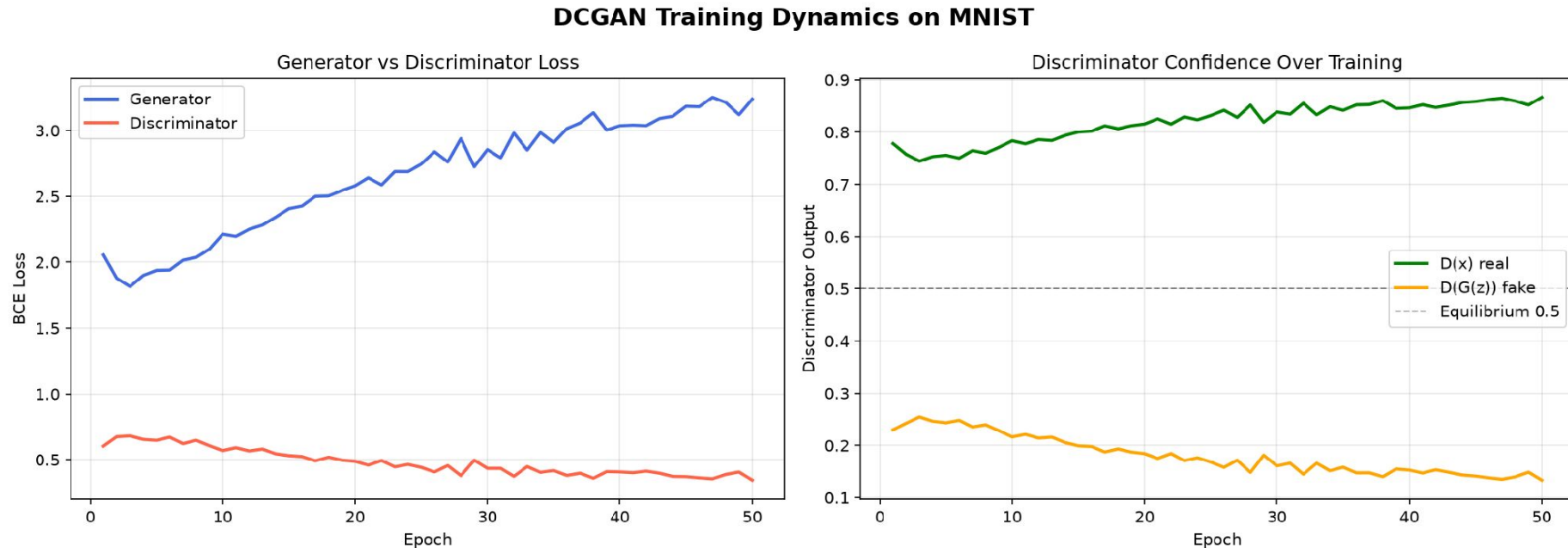
Implementation & Training Setup

Component	Detail
Framework	PyTorch 2.11.0+cu128
Hardware	NVIDIA GeForce RTX 3050 Laptop GPU
Optimiser	Adam (lr = 0.0002)
Loss function	Binary Cross-Entropy (BCELoss)
Epochs	50
Batch size	128
Latent dimension	64

Training design choices

- *Independent Adam optimisers for G and D, so each network updates from its own gradients.*
- *fake_imgs.detach() is used when computing the Discriminator's loss, which stops gradients flowing back into G during D's update step.*
- *A fixed 64-sample noise vector was reused every epoch, so generated samples could be compared side by side as training progressed.*
- *Sample grids saved every 5 epochs- random seed fixed at 42 for reproducibility.*

Training Dynamics



- Over 50 epochs the discriminator held a consistent edge: Its confidence on real images rose from about 0.78 toward 0.85, while its confidence on generated images fell from about 0.23 toward 0.15.
- This pattern is common for a fully-connected generator competing against a comparatively strong discriminator, and it did not stop the generator from producing recognisable digits, as the following slides show.

Generation Quality Across Training

Generated Digits - Epoch 1



Epoch 1

Random noise, no digit structure

Generated Digits - Epoch 25



Epoch 25

Rough digit-like shapes emerging

Generated Digits - Epoch 50



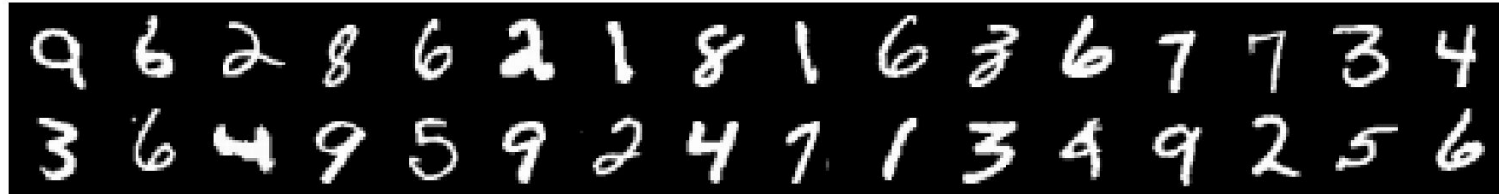
Epoch 50

Recognisable digits across classes

Real vs Generated Digits

Real vs GAN-Generated MNIST Digits

Real MNIST Images



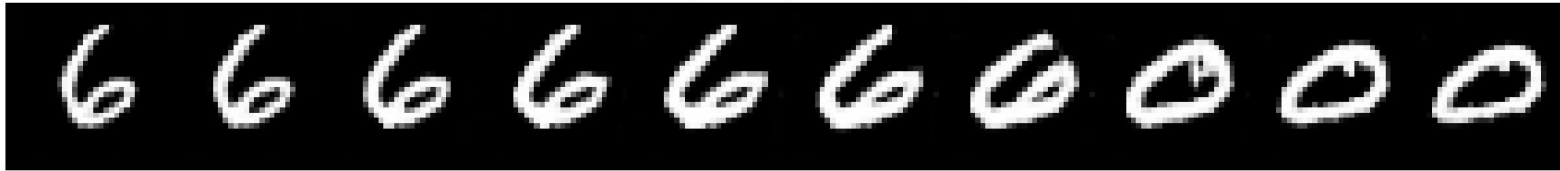
GAN-Generated Images



Generated digits are visually plausible across most classes, though slightly blurrier than real samples. This is typical of MLP-based generators, which do not exploit the spatial locality that convolutional layers capture.

Latent Space Interpolation

Latent Space Interpolation ($z_1 \rightarrow z_2$)



10 interpolation steps $z_1 \rightarrow z_2$

- *Interpolating between two random noise vectors z_1 and z_2 produces a smooth, continuous sequence of plausible digit shapes rather than an abrupt jump.*
- *This is evidence that the Generator learned a structured latent space, instead of simply memorising individual training examples.*

100 Digits Generated From Noise

100 GAN-Generated Handwritten Digits (Final)



- A representative spread across all ten digit classes (0–9), generated purely from random noise.
- Some classes appear more often than others, a mild imbalance rather than full mode collapse.
- A few ambiguous shapes remain, consistent with the blurring typical of MLP-based generators.

Analysis & Interpretation



What Worked

- *Digit-like structure emerged by epoch 10–15 and became clearly recognisable by epoch 50, across multiple classes.*
- *Latent space interpolation confirmed a smooth, continuous internal representation rather than memorisation.*
- *The adversarial setup trained end to end without fully collapsing, and the generator kept improving throughout.*



Limitations

- *No quantitative FID (Fréchet Inception Distance) score was computed, due to compute constraints.*
- *The discriminator kept a persistent edge rather than settling near the theoretical 0.5 equilibrium.*
- *MLP-based generation is blurrier than a convolutional (DCGAN-style) approach would produce.*
- *No class conditioning: the model cannot yet be asked to generate one specific digit.*

09 · CONCLUSION

Conclusion

A fully-connected GAN was implemented and trained on MNIST for 50 epochs, learning to generate visually plausible handwritten digits from 64-dimensional Gaussian noise across all ten digit classes. Latent space interpolation confirmed the model learned a smooth, structured internal representation rather than memorising the training set. The project validates the core GAN principle: two competing networks, trained together, can produce a working generative model.

50

Epochs trained

1.48M

Generator parameters

10 / 10

Digit classes represented

Future Scope

1

Conditional GAN

Condition on the digit class label to generate a specific digit on demand.

2

Wasserstein GAN

Use the Wasserstein distance for more stable training and convergence.

3

FID Score

Compute Fréchet Inception Distance for quantitative quality benchmarking.

4

Convolutional Upgrade

Move to a DCGAN-style architecture for sharper, less blurry images.

5

Data Augmentation

Use generated digits to augment classifiers trained on imbalanced data.

Individual Contributions



Amrith Saras

- Designed and implemented the Generator architecture and full training loop.
- Implemented the latent space interpolation experiment.
- Set up the GPU training environment (CUDA PyTorch).



Diya Jothish

- Verified model saving and loading.
- Designed and implemented the Discriminator architecture and weight initialisation.
- Coded all visualisation functions: epoch grids, loss curves, real vs fake.



Annabel Marianne Victor

- Handled MNIST data loading, normalisation and DataLoader setup.
- Analysed training dynamics and documented the results.
- Verified generated image quality across epochs.

Thank You

Amrith Saras · Annabel Marianne Victor · Diya Jothish

A15 · 102903 / CO701B Deep Learning · Rajagiri School of Engineering & Technology